

C 언어 EXPRESS



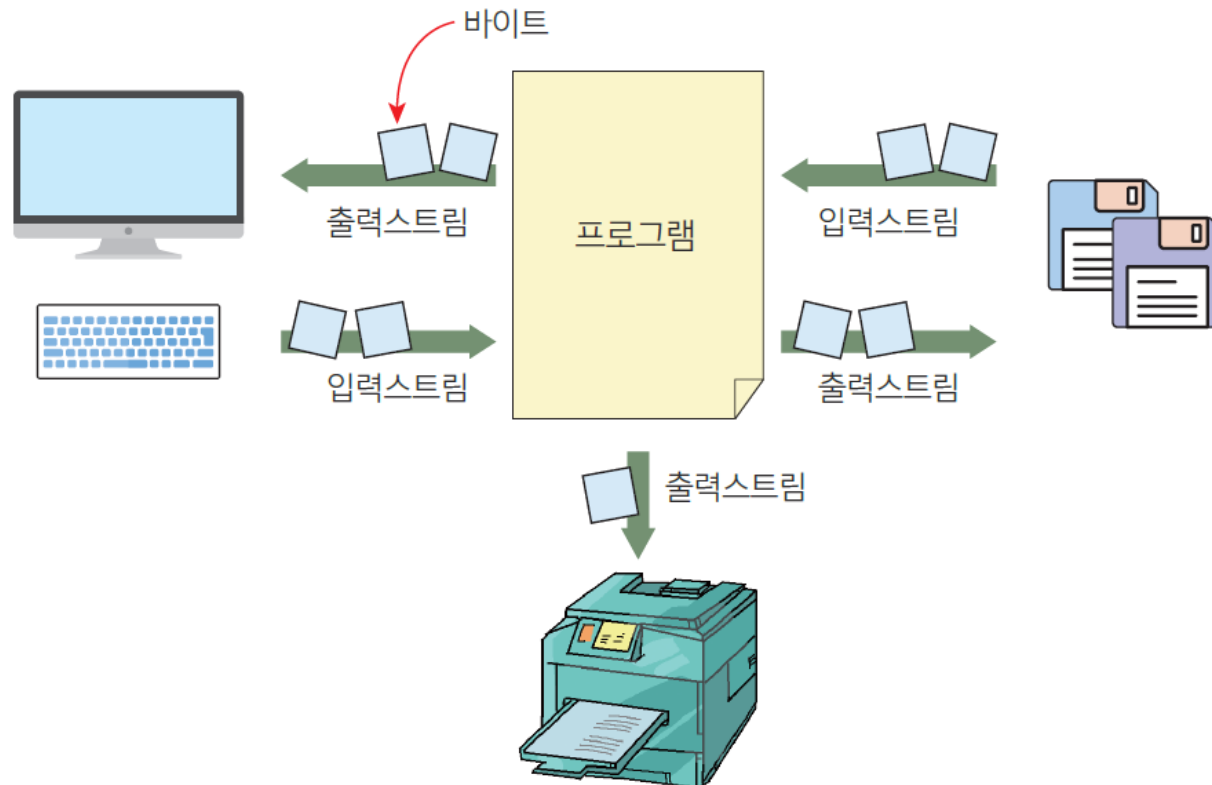
제 15장 파일 입출력



스트림의 개념

2

- 스트림(stream): 입력과 출력을 바이트(byte)들의 흐름으로 생각하는 것

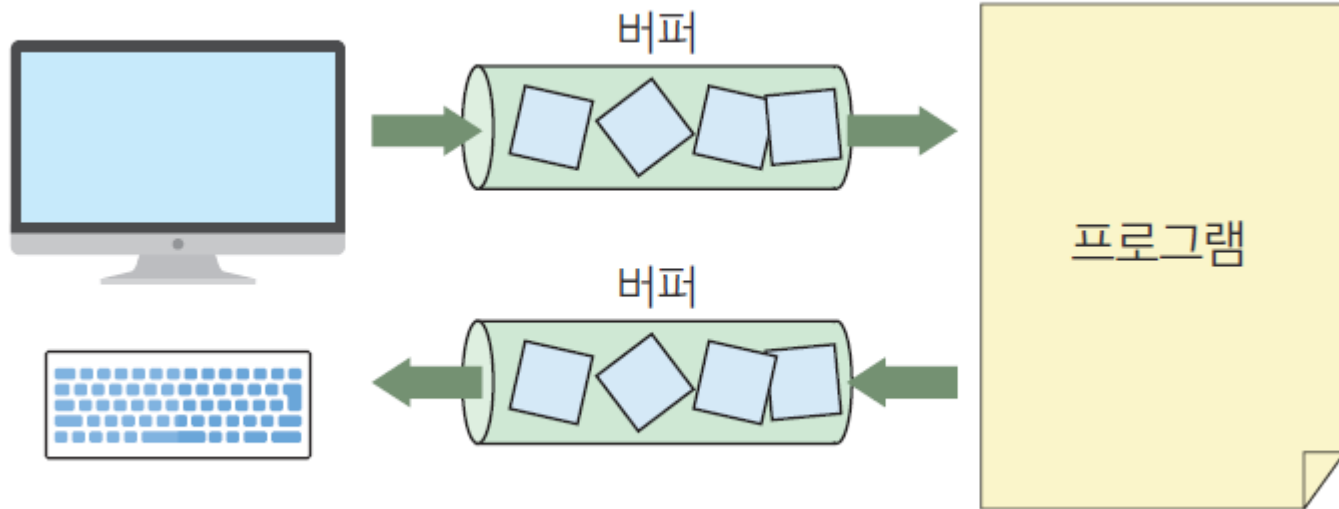




스트림과 버퍼

3

- 스트림에는 기본적으로 버퍼가 포함되어 있다.

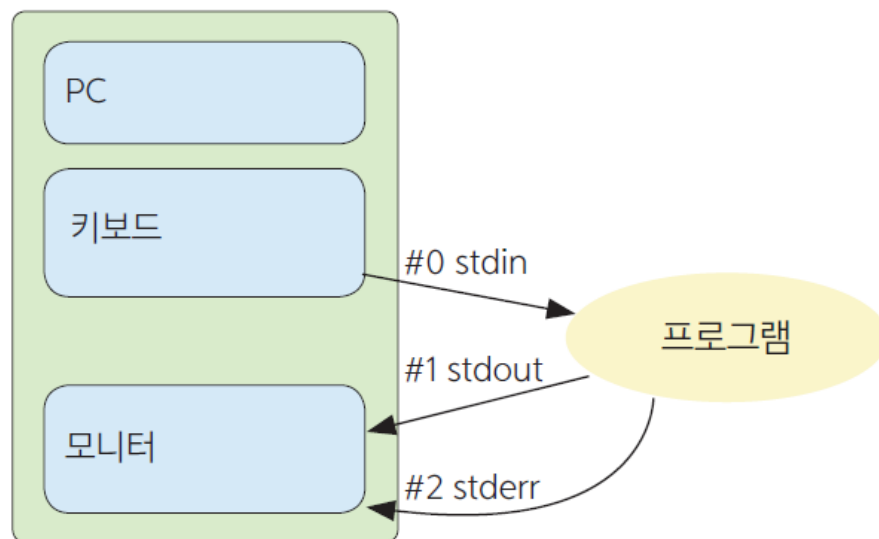




표준 입력 스트림

4

이름	스트림	연결 장치
stdin	표준 입력 스트림	키보드
stdout	표준 출력 스트림	모니터의 화면
stderr	표준 오류 스트림	모니터의 화면





입출력 함수의 종류

5

형식 \ 스트림	표준 스트림	일반 스트림	설명
형식이 없는 입출력 (문자 형태)	getchar()	fgetc(FILE *f,...)	문자 입력 함수
	putchar()	fputc(FILE *f,...)	문자 출력 함수
	gets_s()	fgets(FILE *f,...)	문자열 입력 함수
	puts()	fputs(FILE *f,...)	문자열 출력 함수
형식이 있는 입출력 (정수, 실수...)	printf()	fprintf(FILE *f,...)	형식화된 출력 함수
	scanf()	fscanf(FILE *f,...)	형식화된 입력 함수

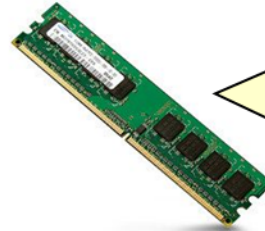


파일이 필요한 이유

6

```
int main(void)
{
    ...
    ...
    ...
}
```

프로그램



메모리

변수, 배열,
구조체 등은
모두 메모리
에 만들이지
고 이것들은
모두 전원이
꺼지면 사라
진다.



하드 디스크

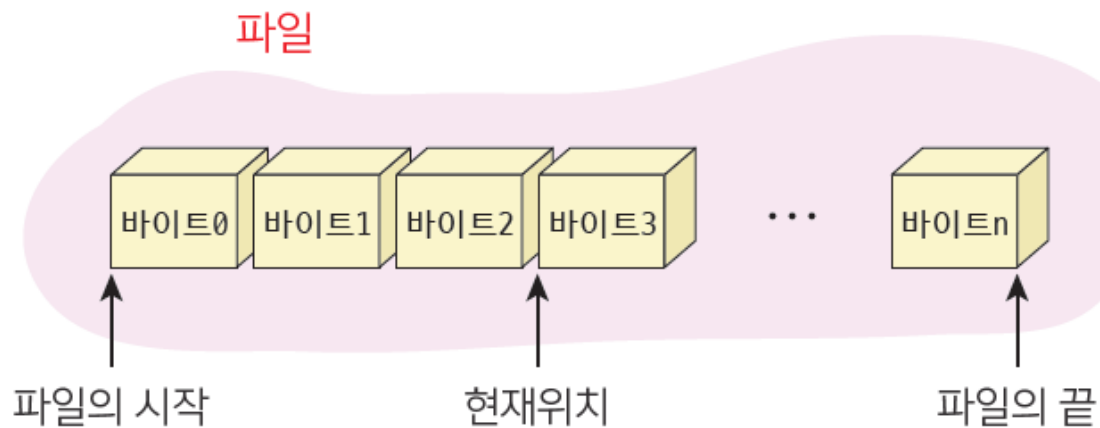
하드 디스크
에 파일 형태
로 저장하면
전원이 꺼지
더라도 데이
터가 보존된
다.



파일의 개념

7

- C에서의 파일은 일련의 연속된 바이트
- 모든 파일 데이터들은 결국은 바이트로 바뀌어서 파일에 저장
- 이들 바이트들을 어떻게 해석하느냐는 전적으로 프로그래머의 책임

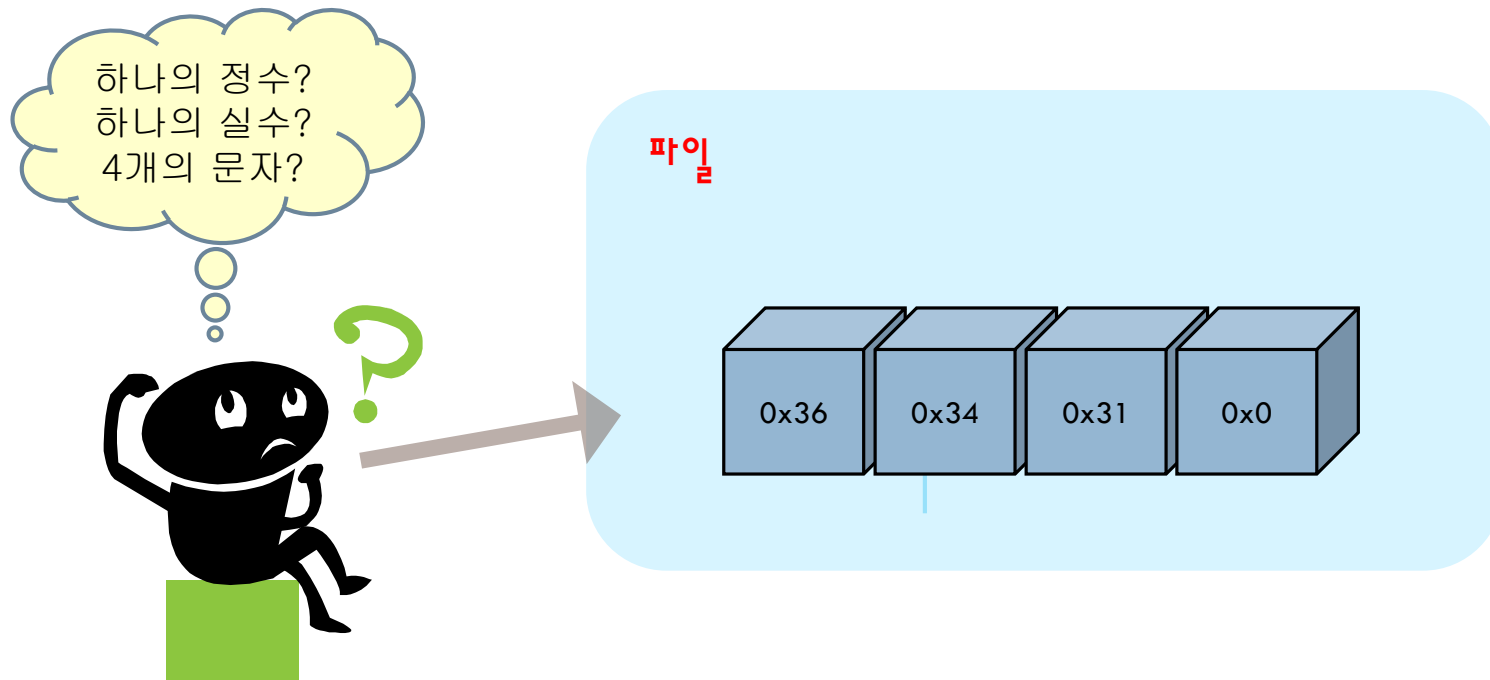




파일

8

- 파일에 4개의 바이트가 들어 있을 때 이것을 **int**형의 정수 데이터로도 해석할 수 있고 아니면 **float**형 실수 데이터로도 해석할 수 있다

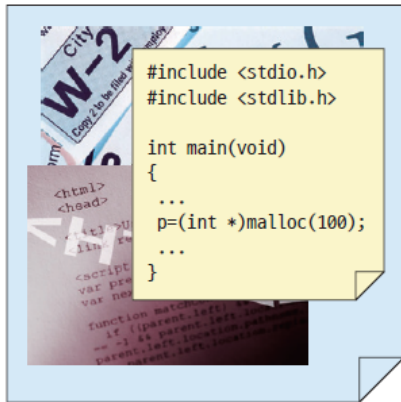




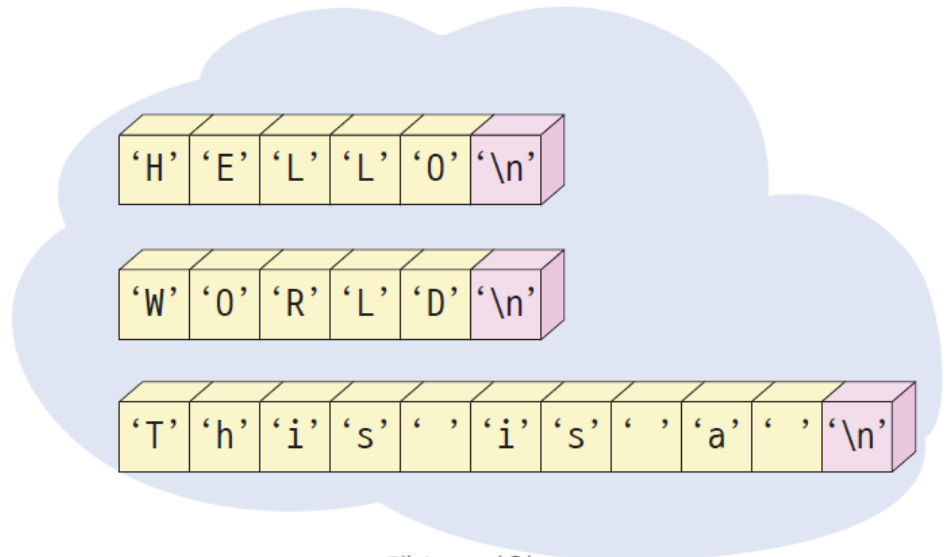
텍스트 파일(text file)

9

- 텍스트 파일은 사람이 읽을 수 있는 텍스트가 들어 있는 파일
 - (예) C 프로그램 소스 파일이나 메모장 파일
- 텍스트 파일은 아스키 코드를 이용하여 저장
- 텍스트 파일은 연속적인 라인들로 구성



텍스트 파일: 문자로 구성된 파일



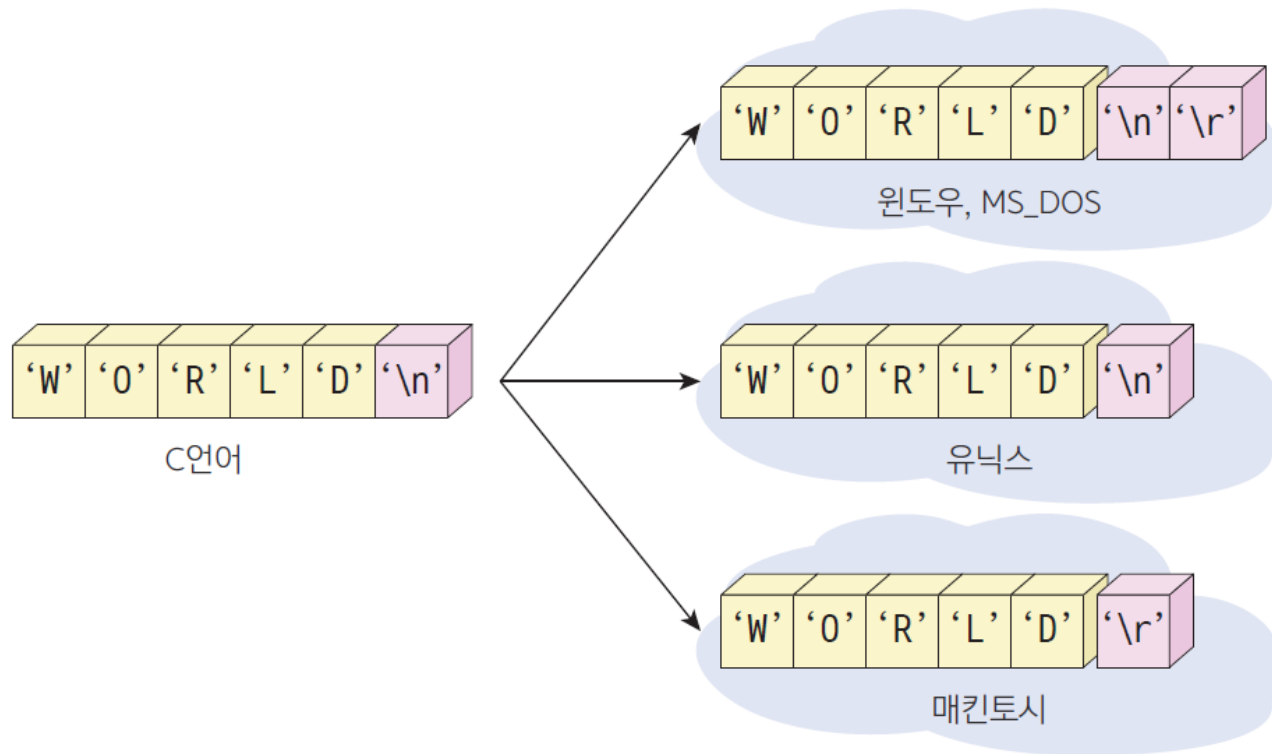
텍스트 파일



텍스트 파일(text file)

10

- 운영체제마다 줄바꿈을 표시하는 방법이 다르다.

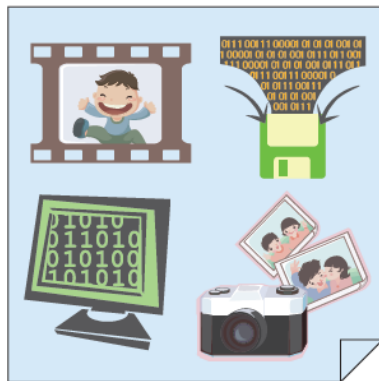




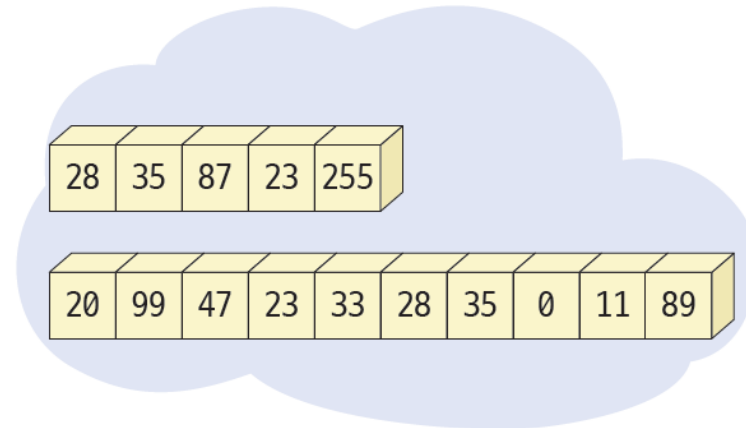
이진 파일(binary file)

11

- 이진 파일은 사람이 읽을 수는 없으나 컴퓨터는 읽을 수 있는 파일
- 이진 데이터가 직접 저장되어 있는 파일
- 이진 파일은 텍스트 파일과는 달리 라인들로 분리되지 않는다.
- 모든 데이터들은 문자열로 변환되지 않고 입출력
- 이진 파일은 특정 프로그램에 의해서만 판독이 가능
- (예) C 프로그램 실행 파일, 사운드 파일, 이미지 파일



이진 파일: 데이터로 구성된 파일



이진 파일



파일 처리의 개요

12

- 파일을 다룰 때는 반드시 다음과 같은 순서를 지켜야 한다.



- 디스크 파일은 **FILE** 구조체를 이용하여 접근
- FILE** 구조체를 가리키는 포인터를 파일 포인터(file pointer)



파일 열기

13

Syntax

파일 열기

예

```
FILE *fp;
```

```
fp = fopen("test.txt", "w");
```

파일 이름

파일 모드



파일 모드

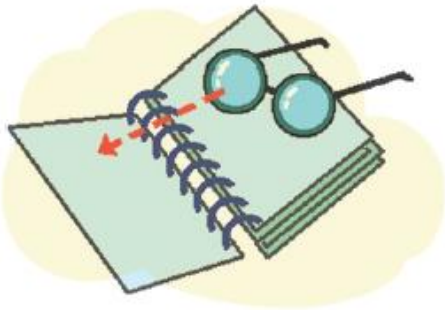
14

모드	설명
"r"	읽기 모드로 파일을 연다. 만약 파일이 존재하지 않으면 오류가 발생한다.
"w"	쓰기 모드로 새로운 파일을 생성한다. 파일이 이미 존재하면 기존의 내용이 지워진다.
"a"	추가 모드로 파일을 연다. 만약 기존의 파일이 있으면 데이터가 파일의 끝에 추가된다. 파일이 없으면 새로운 파일을 만든다.
"r+"	읽기 모드로 파일을 연다. 쓰기 모드로 전환할 수 있다. 파일이 반드시 존재하여야 한다.
"w+"	쓰기 모드로 새로운 파일을 생성한다. 읽기 모드로 전환할 수 있다. 파일이 이미 존재하면 기존의 내용이 지워진다.
"a+"	추가 모드로 파일을 연다. 읽기 모드로 전환할 수 있다. 데이터를 추가하면 EOF 마커를 추가된 데이터의 뒤로 이동한다. 파일이 없으면 새로운 파일을 만든다.
"t"	텍스트 파일 모드로 파일을 연다.
"b"	이진 파일 모드로 파일을 연다.



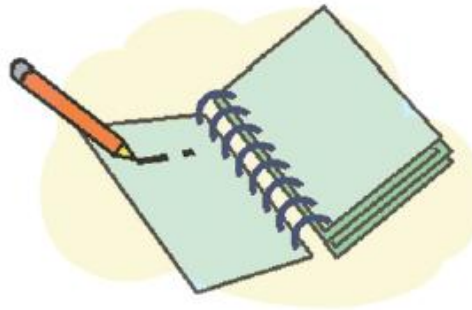
기본적인 파일 모드

15



"r"

파일의 처음 부터 읽는다.



"w"

파일의 처음 부터 쓴다.
만약 파일이 존재하면 기존의
내용이 지워진다.



"a"

파일의 끝에 쓴다.
파일이 없으면 생성 된다.



주의할 점

16

- 기본적인 파일 모드에 "t"나 "b"를 붙일 수 있다.
- "a" 나 "a+" 모드는 **추가 모드(append mode)**라고 한다. 추가 모드로 파일이 열리면, 모든 쓰기 동작은 파일의 끝에서 일어난다. 따라서 파일 안에 있었던 기존의 데이터는 절대 지워지지 않는다.
- "r+", "w+", "a+" 파일 모드가 지정되면 읽고 쓰기가 모두 가능하다. 이러한 모드를 **수정 모드(update mode)**라고 한다. 읽기 모드에서 쓰기 모드로, 또는 쓰기 모드에서 읽기 모드로 전환하려면 반드시 `fflush()`, `fsetpos()`, `fseek()`, `rewind()` 중의 하나를 호출하여야 한다.



파일 닫기

17

Syntax

파일 닫기

예

`fclose(fp):`

FILE 포인터



예제

18

```
#include <stdio.h>
int main(void)
{
    FILE *fp = NULL;

    fp = fopen("sample.txt", "w");

    if( fp == NULL )
        printf("파일 열기 실패\n");
    else
        printf("파일 열기 성공\n");

    fclose(fp);
    return 0;
}
```



파일 열기 성공



파일 삭제 예제

19

```
#include <stdio.h>

int main(void)
{
    if (remove("sample.txt") == -1)
        printf("sample.txt를 삭제 할 수 없습니다.\n");
    else
        printf("sample.txt를 삭제 하였습니다.\n");
    return 0;
}
```

sample.txt를 삭제 하였습니다.



기타 유용한 함수들

20

함수	설명
<code>int foef(FILE *stream)</code>	파일의 끝이 도달되면 true를 반환한다.
<code>int rename(const char *oldname, const char *newname)</code>	파일의 이름을 변경한다.
<code>FILE *tmpfile()</code>	임시 파일을 생성하여 반환한다.
<code>int ferror(FILE *stream)</code>	스트림의 오류 상태를 반환한다. 오류가 발생하면 true가 반환된다.



파일 입출력 함수

21

종류	입력 함수	출력 함수
문자 단위	<code>int fgetc(FILE *fp)</code>	<code>int fputc(int c, FILE *fp)</code>
문자열 단위	<code>char *fgets(char *buf, int n, FILE *fp)</code>	<code>int fputs(const char *buf, FILE *fp)</code>
서식화된 입출력	<code>int fscanf(FILE *fp, ...)</code>	<code>int fprintf(FILE *fp, ...)</code>
이진 데이터	<code>size_t fread(char *buffer, int size, int count, FILE *fp)</code>	<code>size_t fwrite(char *buffer, int size, int count, FILE *fp)</code>



크게 나누면
텍스트 입출력
함수와 이진
데이터
입출력으로 나눌
수 있습니다.



문자 단위 입출력

22

```
#include <stdio.h>
int main(void)
{
    FILE *fp = NULL;

    fp = fopen("sample.txt", "w");
    if( fp == NULL )
        printf("파일 열기 실패\n");
    else
        printf("파일 열기 성공\n");

    fputc('a', fp);
    fputc('b', fp);
    fputc('c', fp);
    fclose(fp);
    return 0;
}
```

파일 열기 성공

sample - 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

abc



문자 단위 입출력

23

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    FILE *fp = NULL;
```

```
    int c;
```

```
    fp = fopen("sample.txt", "r");
```

```
    if( fp == NULL )
```

```
        printf("파일 열기 실패\n");
```

```
    else
```

```
        printf("파일 열기 성공\n");
```

```
    while((c = fgetc(fp)) != EOF )
```

```
        putchar(c);
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

정수형 변수로 선언하여야 한다.

파일 열기 성공
abc

sample - 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

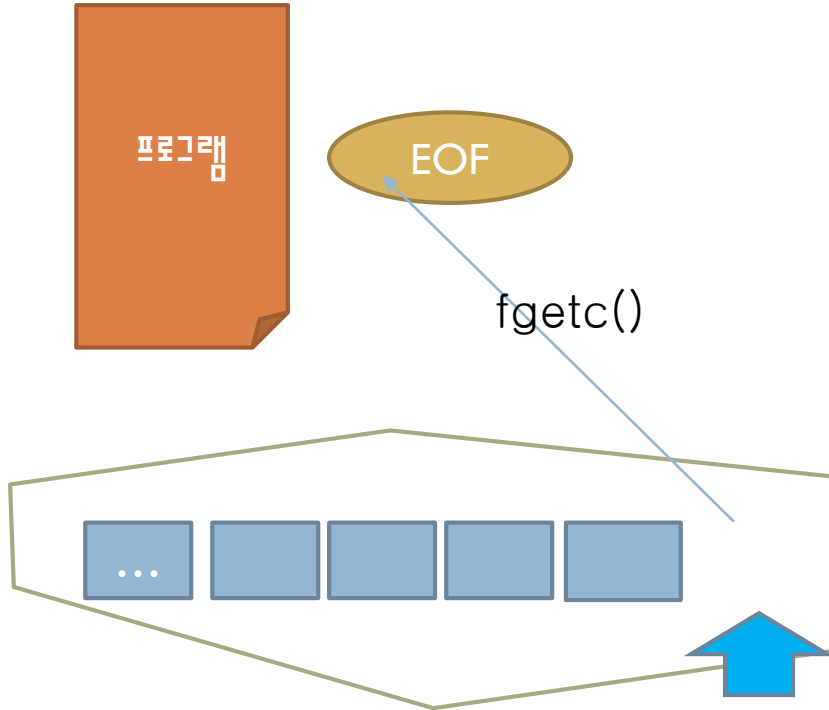
abcd



EOF

24

- EOF(End Of File): 파일의 끝을 나타내는 특수한 기호
 - 정수 -1 값





문자열 단위 입출력

25

Syntax

문자열 단위 입출력

예

여기에 문자열을 저장

최대 개수

```
char *fgets( char *s, int n, FILE *fp );  
int fputs( char *s, FILE *fp );
```



문자열 다위 입출력

26

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void)
```

```
{
```

```
    FILE *fp1, *fp2;
```

```
    char file1[100], file2[100];
```

```
    char buffer[100];
```

```
    printf("원본 파일 이름: ");
```

```
    scanf("%s", file1);
```

```
    printf("복사 파일 이름: ");
```

```
    scanf("%s", file2);
```

```
    // 첫번째 파일을 읽기 모드로 연다.
```

```
    if( (fp1 = fopen(file1, "r")) == NULL )
```

```
    {
```

```
        fprintf(stderr, "원본 파일 %s을 열 수 없습니다.\n", file1);
```

```
        exit(1);
```

```
    }
```



문자열 다위 입출력

27

```
// 두번째 파일을 쓰기 모드로 연다.  
if( (fp2 = fopen(file2, "w")) == NULL )  
{  
    fprintf(stderr, "복사 파일 %s을 열 수 없습니다.\n", file2);  
    exit(1);  
}  
  
// 첫번째 파일을 두번째 파일로 복사한다.  
while( fgets(buffer, 100, fp1) != NULL )  
    fputs(buffer, fp2);  
  
fclose(fp1);  
fclose(fp2);  
  
return 0;  
}
```

원본 파일 이름: a.txt
복사 파일 이름: b.txt



형식화된 입출력

28

Syntax

형식화된 입출력

예

```
int fprintf( FILE *fp,  const char *format, ...);  
int fscanf( FILE *fp,  const char *format, ...);
```



예제

29

```
int main(void)
{
    FILE *fp;
    char fname[100];
    int number, count = 0;
    char name[20];
    float score, total = 0.0;

    printf("성적 파일 이름을 입력하시오: ");
    scanf("%s", fname);

    // 성적 파일을 쓰기 모드로 연다.
    if( (fp = fopen(fname, "w")) == NULL )
    {
        fprintf(stderr, "성적 파일 %s을 열 수 없습니다.\n", fname);
        exit(1);
    }
}
```



예제

30

```
// 사용자로부터 학번, 이름, 성적을 입력받아서 파일에 저장한다.
while( 1 )
{
    printf("학번, 이름, 성적을 입력하시요: (음수이면 종료)");
    scanf("%d", &number);
    if( number < 0 ) break
    scanf("%s %f", name, &score);
    fprintf(fp, " %d %s %f", number, name, score);
}
fclose(fp);
// 성적 파일을 읽기 모드로 연다.
if( (fp = fopen(fname, "r")) == NULL )
{
    fprintf(stderr, "성적 파일 %s을 열 수 없습니다.\n", fname);
    exit(1);
}
```



예제

31

```
// 파일에서 성적을 읽어서 평균을 구한다.  
while( !feof( fp ) )  
{  
    fscanf(fp, "%d %s %f", &number, name, &score);  
    total += score;  
    count++;  
}  
printf("평균 = %f\n", total/count);  
fclose(fp);  
return 0;  
}
```

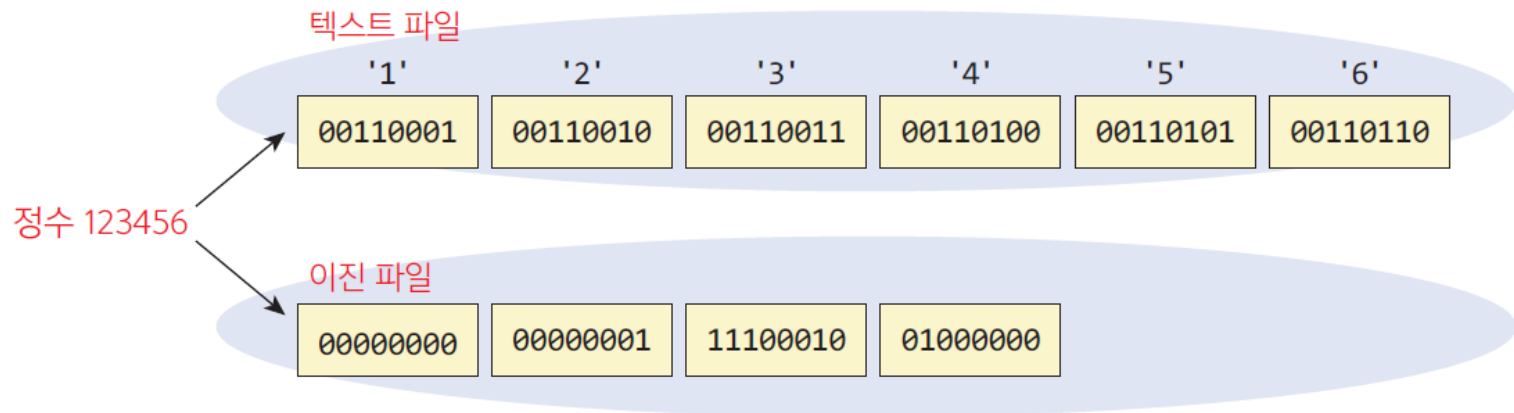
성적 파일 이름을 입력하시오: scores.txt
학번, 이름, 성적을 입력하시요: (음수이면 종료)1 KIM 10.0
학번, 이름, 성적을 입력하시요: (음수이면 종료)2 PARK 20.0
학번, 이름, 성적을 입력하시요: (음수이면 종료)3 LEE 30.0
학번, 이름, 성적을 입력하시요: (음수이면 종료)-1
평균 = 20.000000



이진 파일 쓰기과 읽기

32

- 텍스트 파일과 이진 파일의 차이점
 - **텍스트 파일**: 모든 데이터가 아스키 코드로 변환되어서 저장됨
 - **이진 파일**: 컴퓨터에서 데이터를 표현하는 방식 그대로 저장

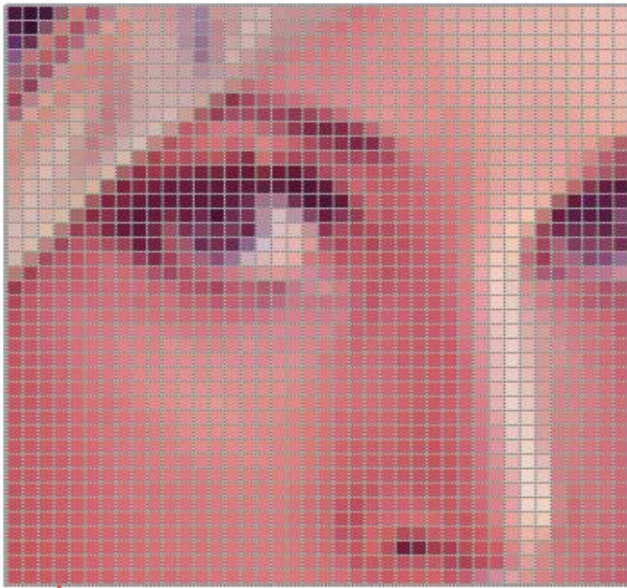




이진 파일의 예

33

- 이미지 파일이나 사운드 파일



각 픽셀의 밝기를 이진수로
나타낸다.



이진 파일 모드

34

파일 모드	설명
"rb"	읽기 모드 + 이진 파일 모드
"wb"	쓰기 모드 + 이진 파일 모드
"ab"	추가 모드 + 이진 파일 모드
"rb+"	읽고 쓰기 모드 + 이진 파일 모드
"wb+"	쓰고 읽기 모드 + 이진 파일 모드



이진 파일 쓰기

35

Syntax

fwrite()

예

```
fwrite(buffer, sizeof(int), SIZE, fp);
```

메모리 블록의 주소

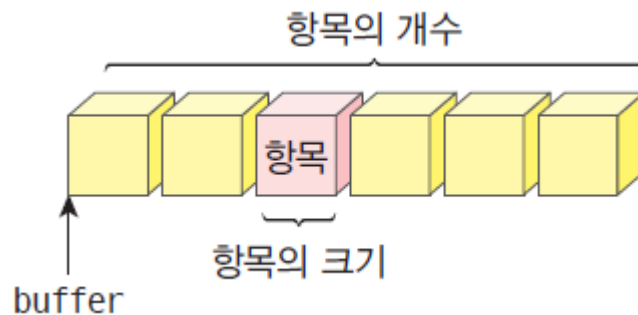
항목의 크기

항목의 개수

FILE 포인터

fwrite(buffer, size, count, fp)

- buffer는 파일에 기록할 데이터를 가지고 있는 메모리 블록의 시작 주소이다.
- size는 저장되는 항목의 크기로서 단위는 바이트이다.
- count는 저장하려고 하는 항목의 개수이다. 만약 int형의 데이터 10개를 쓰려고 하면 항목의 크기는 4가 되고 항목의 개수는 10이 될 것이다.
- fp는 FILE 포인터이다.





이진 파일 쓰기

36

```
#include <stdio.h>

#define SIZE 5
int main(void)
{

    int buffer[SIZE] = { 10, 20, 30, 40, 50 };
    FILE* fp = NULL;

    fp = fopen("binary.bin", "wb");    // ①
    if (fp == NULL)
    {
        fprintf(stderr, "binary.bin 파일을 열 수 없습니다.");
        return 1;
    }

    fwrite(buffer, sizeof(int), SIZE, fp);    // ②

    fclose(fp);
    return 0;
}
```



이진 파일 읽기

37

Syntax

fread()

예

```
fread(buffer, sizeof(int), SIZE, fp);
```

메모리 블록의 주소

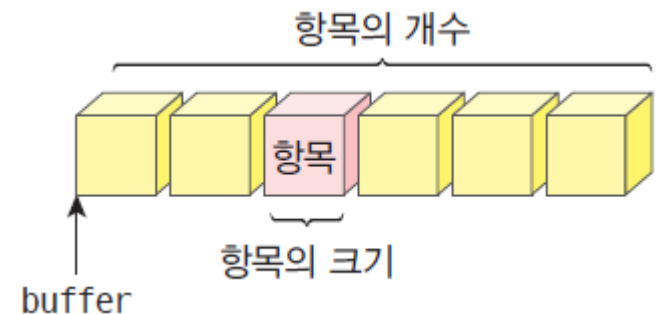
항목의 크기

항목의 개수

FILE 포인터

fread(buffer, size, count, fp)

- buffer는 파일에서 데이터를 읽어 기록할 메모리 블록의 시작 주소이다.
- size는 저장되는 항목의 크기로서 단위는 바이트이다.
- count는 저장하려고 하는 항목의 개수이다. 만약 int형의 데이터 10개를 쓰려고 하면 항목의 크기는 4가 되고 항목의 개수는 10이 될 것이다.
- fp는 FILE 포인터이다.





이진 파일 읽기

38

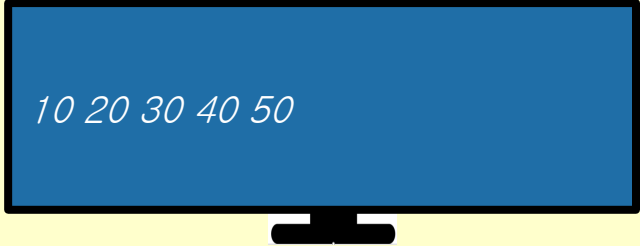
```
#include <stdio.h>
#define SIZE 5

int main(void)
{
    int i;
    int buffer[SIZE];
    FILE* fp = NULL;

    fp = fopen("binary.bin", "rb");
    if (fp == NULL)
    {
        fprintf(stderr, "binary.bin 파일을 열 수 없습니다.");
        return 1;
    }
    fread(buffer, sizeof(int), SIZE, fp);

    for (i = 0; i < SIZE; i++)
        printf("%d ", buffer[i]);

    fclose(fp);
    return 0;
}
```



10 20 30 40 50



Lab: 이진 파일에 학생 정보 저장하기

39

- 학생들의 정보를 이진 파일에 저장하고 다시 읽어서 화면에 출력하는 프로그램을 작성하여 보자.

```
학번 = 1, 이름 = Kim, 평점 = 3.99  
학번 = 2, 이름 = Min, 평점 = 2.68  
학번 = 3, 이름 = Lee, 평점 = 4.01
```



예제

40

```
#define SIZE 3
struct student {
    int number;           // 학번
    char name[20]; // 이름
    double gpa;           // 평점
};

int main(void)
{
    struct student table[SIZE] = {
        { 1, "Kim", 3.99 },
        { 2, "Min", 2.68 },
        { 3, "Lee", 4.01 }
    };
    struct student s;
    FILE *fp = NULL;
    int i;
    // 이진 파일을 쓰기 모드로 연다.
    if( (fp = fopen("student.dat", "wb")) == NULL )
    {
        fprintf(stderr, "출력을 위한 파일을 열 수 없습니다.\n");
        exit(1);
    }
}
```




예제

41

```
// 배열을 파일에 저장한다.
fwrite(table, sizeof(struct student), SIZE, fp);
fclose(fp);
// 이진 파일을 읽기 모드로 연다.
if( (fp = fopen("student.dat", "rb")) == NULL )
{
    fprintf(stderr, "입력을 위한 파일을 열 수 없습니다.\n");
    exit(1);
}
for(i = 0; i < SIZE; i++)
{
    fread(&s, sizeof(struct student), 1, fp);
    printf("학번 = %d, 이름 = %s, 평점 = %.2f\n", s.number, s.name, s.gpa);
}
fclose(fp);
return 0;
}
```

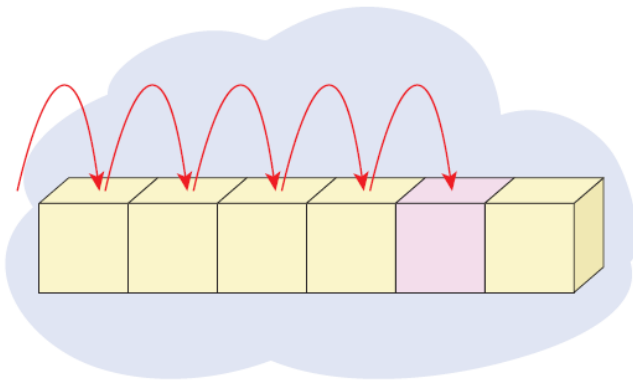
```
학번 = 1, 이름 = Kim, 평점 = 3.99
학번 = 2, 이름 = Min, 평점 = 2.68
학번 = 3, 이름 = Lee, 평점 = 4.01
```



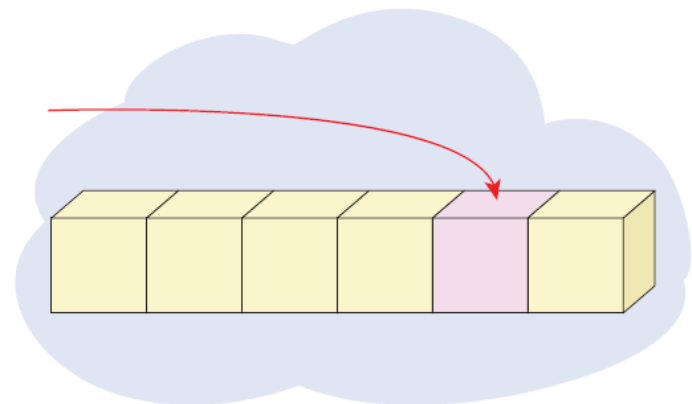
임의 접근 파일

42

- **순차 접근(sequential access)** 방법: 데이터를 파일의 처음부터 순차적으로 읽거나 기록하는 방법
- **임의 접근(random access)** 방법: 파일의 어느 위치에서든지 읽기와 쓰기가 가능한 방법



순차 접근파일



임의 접근파일

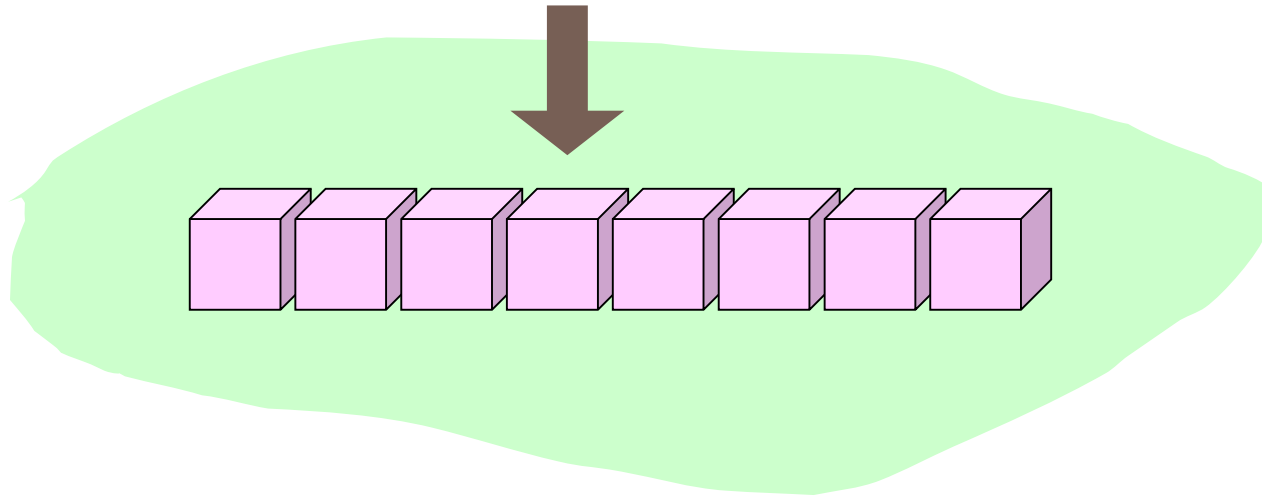


임의 접근 파일의 위치

43

- 파일 위치 지정자(file position indicator) : 읽기와 쓰기 동작이 현재 어떤 위치에서 이루어지는 지를 나타낸다.

파일 위치 지정자



- 강제로 파일 위치 지정자를 이동시키면 임의 접근이 가능



fseek()

44

Syntax

fseek()

예

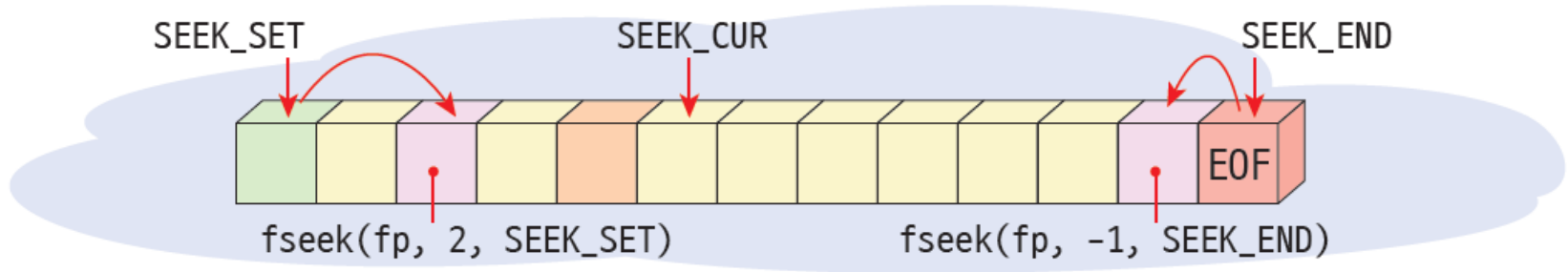
`int` fseek(FILE 포인터 `FILE *fp`, 거리 `long offset`, 기준 위치 `int origin`);

상수	값	설명
SEEK_SET	0	파일의 시작
SEEK_CUR	1	현재 위치
SEEK_END	2	파일의 끝



fseek()

45



- `rewind(fp)`: 파일 위치 지정자를 첫 부분으로 초기화한다.



ftell(), feof()

46

Syntax: ftell()

예

`long ftell(FILE *fp);`

파일 위치 지정자의 현재의 위치를 반환한다.

Syntax: feof()

예

`int feof(FILE *fp);`

파일의 끝에 도달하였는지 여부를 반환한다.



Q & A

47

